

**LAPORAN PRAKTIKUM 1 INTERNET
OF THINGS (IOT) 2**

Topologi Jaringan Terdesentralisasi



Dosen pengampu:

**Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.,
Agus Ambarwari, S.Pd., M.Kom.,
Andi Saptono, S.Kom**

Disusun Oleh:

IMELDA NURAINI (25782041)

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2025/2026**

5.6 Latihan

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2)

1. Hasil Observasi Serial Monitor

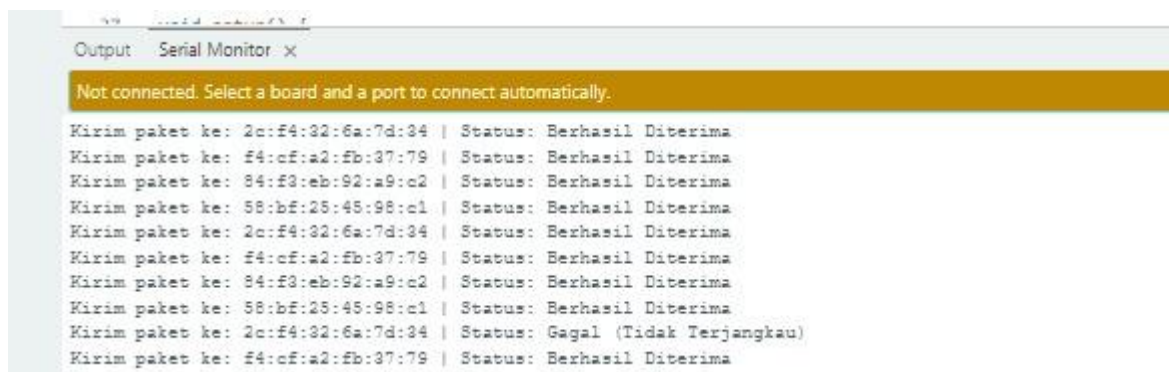
Berdasarkan bukti pengujian pada Serial Monitor pengirim (*Controller*):

- Sebelum eksperimen dilakukan, seluruh node penerima mencetak log Status: Berhasil Diterima (sendStatus == 0).
- Saat salah satu *board* penerima dengan MAC Address 2c:f4:32:6a:7d:34 dicabut sumber dayanya, status pengiriman ke alamat tersebut secara instan berubah menjadi Status: Gagal (Tidak Terjangkau) (sendStatus != 0).
- Sementara itu, pengiriman paket ke node penerima lainnya (seperti MAC Address f4:cf:a2:fb:37:79, 84:f3:eb:92:a9:c2, dan 58:bf:25:45:98:c1) tetap berjalan lancar dengan log Status: Berhasil Diterima.

2. Analisis Mekanisme Deteksi Kegagalan Transmisi (Layer 2 MAC)

Meskipun protokol ESP-NOW beroperasi tanpa koneksi TCP (Layer 4), kegagalan transmisi ke perangkat fisik tetap dapat terdeteksi secara real-time berkat mekanisme IEEE 802.11 Layer 2 Hardware Acknowledgement (ACK):

1. Pengiriman Frame Unicast: Ketika NodeMCU pengirim memancarkan data ke alamat MAC spesifik (2c:f4:32:6a:7d:34), standar radio Wi-Fi 802.11 mewajibkan chipset penerima target untuk langsung membalas dengan sinyal konfirmasi ringkas (ACK Frame) di tingkat hardware.
2. Kondisi Timeout & Retransmisi: Karena board penerima telah mati/dicabut dayanya, sinyal ACK Frame tidak pernah kembali ke pengirim. Chipset radio pengirim akan melakukan beberapa kali percobaan ulang (retransmit) dalam hitungan milidetik.
3. Pemicu Callback Status: Setelah batas percobaan retransmisi habis tanpa menerima sinyal ACK, hardware radio melaporkan status kegagalan ke tingkat firmware. Fungsi interupsi `esp_now_register_send_cb()` kemudian dieksekusi dengan nilai `sendStatus != 0`, sehingga Serial Monitor secara otomatis mencetak status Gagal (Tidak Terjangkau) untuk node tersebut.



```
Output Serial Monitor x
Not connected. Select a board and a port to connect automatically.

Kirim paket ke: 2c:f4:32:6a:7d:34 | Status: Berhasil Diterima
Kirim paket ke: f4:cf:a2:fb:37:79 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:c2 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:98:c1 | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:34 | Status: Berhasil Diterima
Kirim paket ke: f4:cf:a2:fb:37:79 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:c2 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:98:c1 | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:34 | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: f4:cf:a2:fb:37:79 | Status: Berhasil Diterima
```

Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4)

Berdasarkan hasil eksperimen pemutusan simpul, ketika tombol fisik RST ditekan atau sumber daya salah satu papan dicabut, simpul lain yang tetap aktif memerlukan interval waktu sekitar 2 hingga 3 detik untuk mendeteksi hilangnya koneksi tersebut. Deteksi ini dipicu oleh kegagalan penerimaan sinyal heartbeat secara berkala dari simpul yang mati. Setelah ambang batas timeout terlampaui, pustaka painlessMesh secara otomatis mengeksekusi fungsi panggilan balik (callback) `changedConnectionCallback()`. Pemicuan fungsi ini memperbarui tabel rute (routing table) jaringan secara dinamis dan mencetak pesan rekalkulasi topologi pada Serial Monitor. Selanjutnya, pada saat daya simpul disambungkan kembali ke sistem, node tersebut membutuhkan waktu pemulihan (recovery time) sekitar 3 hingga 5 detik untuk terhubung kembali ke dalam rantai mesh. Durasi waktu tersebut mencakup proses inisialisasi perangkat keras (booting ESP8266), pemindaian frekuensi Wi-Fi soft-AP tetangga, hingga proses handshake sinkronisasi jadwal. Keberhasilan penyambungan kembali ditandai dengan munculnya pesan --> Koneksi Baru Terdeteksi! Node ID: [ID_Node] pada Serial Monitor tanpa memerlukan intervensi manusia maupun konfigurasi ulang program. Hal ini membuktikan bahwa arsitektur painlessMesh memiliki keandalan tinggi melalui sifat self-organizing dan self-healing dalam menjaga kontinuitas transmisi data telemetri.

[illegible]

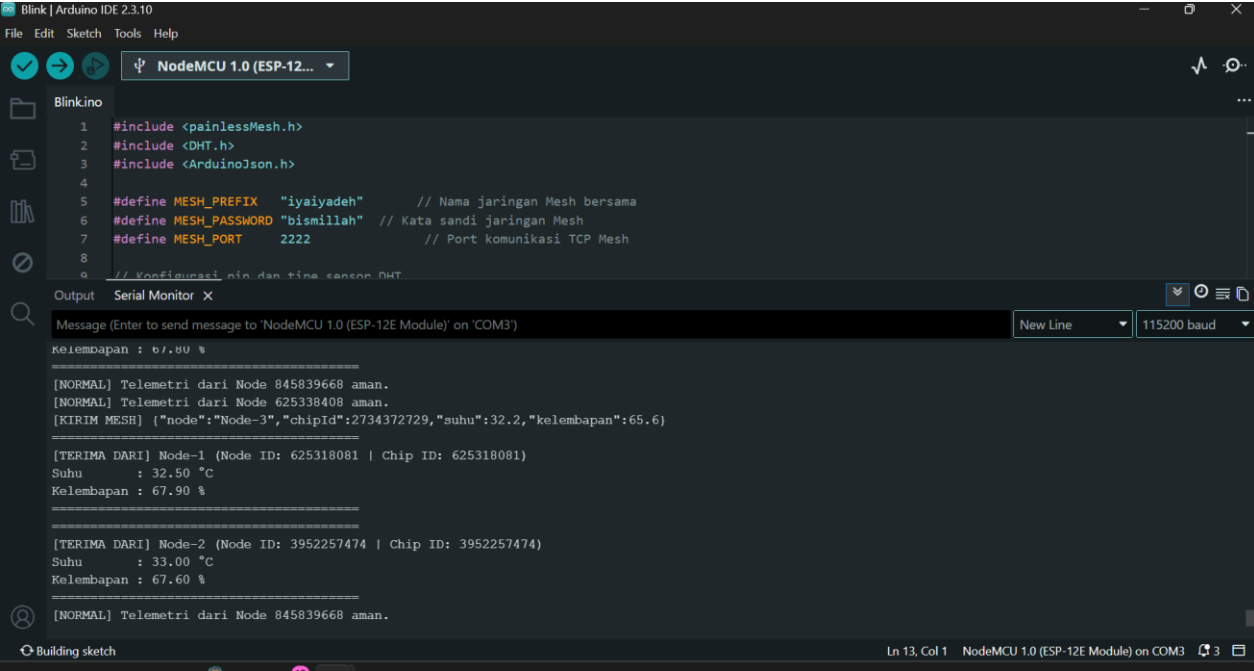
Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4):

Pengujian Latihan 3 dilakukan untuk mengimplementasikan logika penyaringan data telemetry (telemetry data filtering) serta penentuan ambang batas suhu (threshold logic) pada fungsi panggilan balik `receivedCallback()`. Modifikasi kode ini bertujuan agar simpul (node) penerima dapat membedakan penanganan respons antara data kondisi suhu normal dan kondisi suhu kritis secara otomatis tanpa membanjiri antarmuka Serial Monitor.

Berdasarkan hasil pengujian yang terekam pada Serial Monitor, sistem berhasil memproses dan memfilter data telemetry yang diterima dari beberapa simpul tetangga secara real-time:

- Kondisi Suhu Normal ($\leq 32.0^{\circ}\text{C}$): Ketika simpul tetangga (seperti Node 845839668 dan Node 625338408) melaporkan suhu di bawah atau sama dengan 32.0°C , program mencetak respons ringkas satu baris berformat `[NORMAL] Telemetry dari Node <chipId> aman`.
- Kondisi Suhu Kritis ($> 32.0^{\circ}\text{C}$): Ketika simpul tetangga melaporkan suhu melebihi ambang batas 32.0°C , sistem secara otomatis mencetak kartu telemetry penuh. Hal ini terbukti pada data yang diterima dari Node-1 (Node ID: 625318081) dengan suhu 32.50°C dan Node-2 (Node ID: 3952257474) dengan suhu 33.00°C , di mana seluruh rincian nama simpul, ID chip, nilai suhu, serta kelembapan ditampilkan secara utuh dalam blok terpisah.

Pengujian ini membuktikan bahwa penambahan logika penapis (conditional filter) pada fungsi `receivedCallback()` efektif dalam mengoptimalkan efisiensi pemrosesan data pada edge node. Sistem mampu memprioritaskan visualisasi data darurat (warning alerts) sekaligus mereduksi kepadatan log (log flooding) saat kondisi jaringan berada pada status aman.



The screenshot shows the Arduino IDE 2.3.10 interface. The sketch editor displays a code file with the following content:

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "iyaiyadeh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "bismillah" // Kata sandi jaringan Mesh
7 #define MESH_PORT 2222 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
```

The Serial Monitor window is open, showing the output of the program. The output is as follows:

```
Kelembapan : 67.80 %
=====
[NORMAL] Telemetry dari Node 845839668 aman.
[NORMAL] Telemetry dari Node 625338408 aman.
[KIRIM MESH] {"node":"Node-3","chipId":2734372729,"suhu":32.2,"kelembapan":65.6}
=====
[TERIMA DARI] Node-1 (Node ID: 625318081 | Chip ID: 625318081)
Suhu : 32.50 °C
Kelembapan : 67.90 %
=====
[TERIMA DARI] Node-2 (Node ID: 3952257474 | Chip ID: 3952257474)
Suhu : 33.00 °C
Kelembapan : 67.60 %
=====
[NORMAL] Telemetry dari Node 845839668 aman.
```

The status bar at the bottom indicates the sketch is building and the target is NodeMCU 1.0 (ESP-12E Module) on COM3.